Hospital Management System Project Report

---

1. Student Details

Name: Sarthak Kumar Tiwari
Roll Number: 23f3000340
Email: 23f3000340@ds.study.iitm.ac.in
About Me: I am passionate about full-stack web development and data-driven healthcare solutions. I enjoy building scalable applications that improve user experience and streamline hospital operations.

---

2. Project Details

Project Title: Hospital Management System

Problem Statement:
To design and build a web-based hospital management system that enables efficient management of doctors, patients, appointments, treatments, and medical notes. The system should support role-based access, appointment scheduling, and data analytics for hospital administration.

Approach:
The app uses Flask for the backend and Vue.js for the frontend. The backend is modular, with SQLAlchemy models for all entities. RESTful APIs allow CRUD operations for users, doctors, patients, appointments, treatments, and notes. The frontend provides interactive dashboards and forms for all user roles.

---

3. AI/LLM Declaration

The extent of AI/LLM usage is around 10–15%, limited to code debugging, styling , brainstorming and documentation.

4. Technologies and Frameworks Used

Technology / Library	**Purpose**
Flask	Core backend web framework
SQLAlchemy	ORM for SQLite database
Flask-Security	User authentication and role management
Celery + Redis	Background task processing (e.g., email reports)
Vue.js	Frontend SPA framework
Bootstrap 5	Responsive UI styling
SQLite	Local database for all entities

---

5. Database Schema / ER Diagram

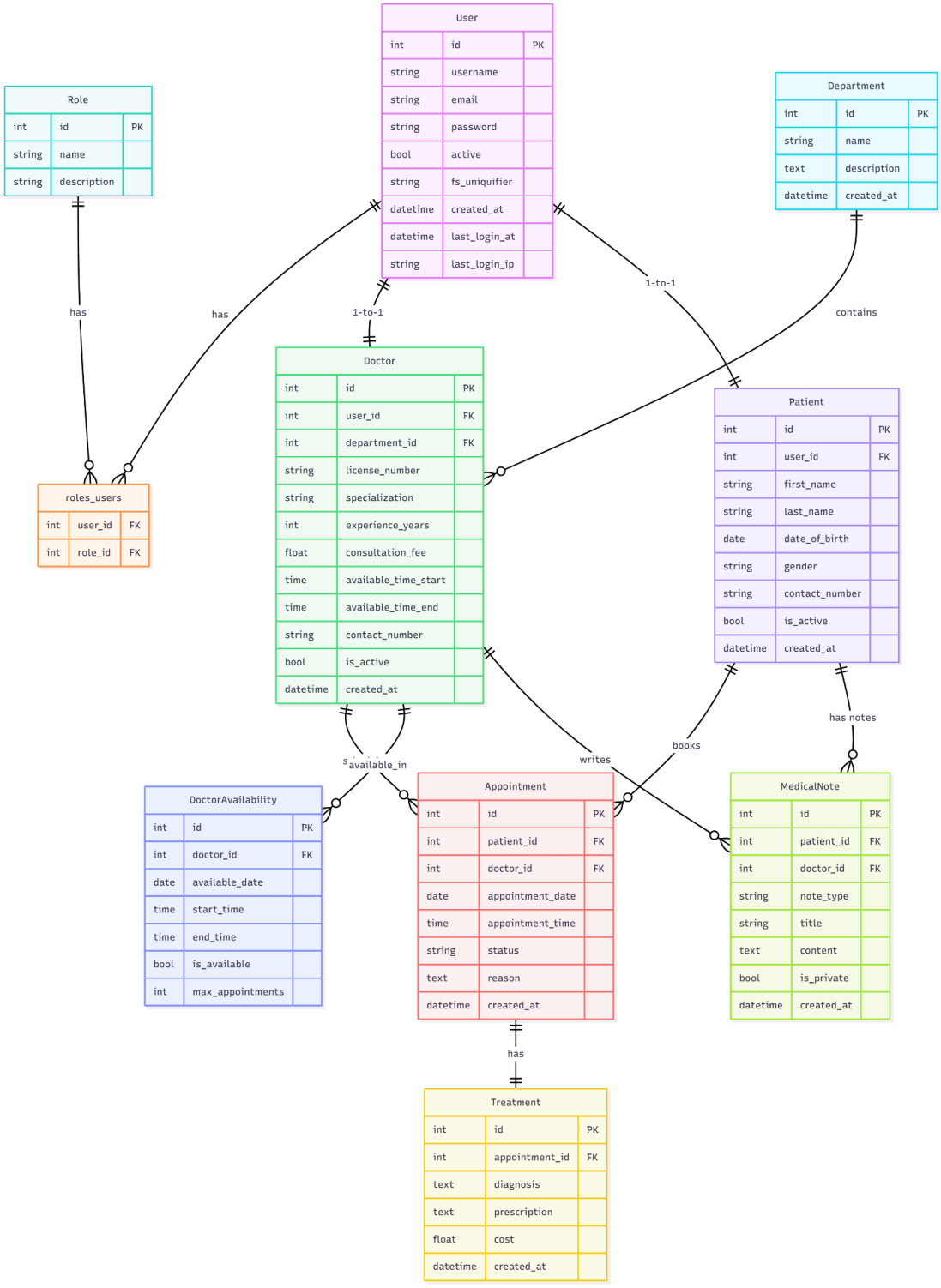
Tables:

- **User** — stores user profile and authentication details (id, username, email, password, roles)
- **Role** — user roles (id, name, description)
- **Department** — hospital departments (id, name, description)
- **Doctor** — doctor profiles (id, user_id, department_id, specialization, etc.)
- **Patient** — patient profiles (id, user_id, personal info, medical history)
- **Appointment** — appointment records (id, patient_id, doctor_id, date, time, status)
- **Treatment** — treatment details linked to appointments (id, appointment_id, diagnosis, prescription)
- **DoctorAvailability** — doctor available slots (id, doctor_id, date, time)
- **MedicalNote** — clinical notes by doctors (id, patient_id, doctor_id, title, content)

Relationships:

- One-to-Many: Department → Doctor
- One-to-Many: Doctor → Appointment
- One-to-Many: Patient → Appointment
- One-to-One: Appointment → Treatment

- Many-to-Many: User ↔ Role



6. API Resource Endpoints

Endpoint	Method	Description
/api/login	POST	Authenticate user and generate session
/api/users	GET	Fetch user profiles
/api/doctors	GET	List all doctors
/api/patients	GET	List all patients
/api/appointments	GET	Fetch appointments
/api/appointment/add	POST	Add new appointment
/api/treatments	GET	List treatments
/api/medical_notes	GET	List medical notes
/api/doctor_availability	GET	Get doctor availability slots
/api/summary	GET	Return analytics summary (admin dashboard)

YAML API Definition File:
Included separately in the submission ZIP as `api.yaml`.

---

7. Architecture and Features

Architecture Overview:

- `backend/app.py` – main Flask application entry point
- `backend/models.py` – SQLAlchemy models for all entities
- `backend/resources.py` – API resource definitions
- `backend/tasks.py` – Celery background tasks
- `frontend/` – Vue.js SPA (components, views, store, router)
- `backend/templates/` – Jinja2 HTML templates (if used)
- `backend/static/` – CSS, JS, and

Implemented Features:

- User registration, login, and role-based access
- Doctor and patient management
- Appointment scheduling and status tracking
- Treatment logging and medical notes
- Responsive UI with Bootstrap
- Secure password hashing and session management
- Email notifications (via Celery)

Additional Features:

- Export data as CSV
- Automated monthly doctor report emails
- Blacklisting of doctors/patients

---

8. Video Presentation

Drive Link:

https://drive.google.com/file/d/1o-DBjQbwTNtmcl8P_eTz7W6z0CvroOud/view?usp=sharing